

CSCI 232

232 Algorithms & Data Structures

LECTURE: 18

J. L. Pach

Today's Agenda

- Binary Trees
  - Why trees? Terminology, the recursive definition
  - From `DoubleNode` to `TreeNode`: left, right and **parent** pointer
- Exceptions
  - The C convention: return `0` / `-1`
  - `throw`, `try`, `catch`, stack unwinding, standard exception types
  - C++ is not Java or C#. Who is responsible for correct data?
- Putting It Together: the `BinaryTree` Wrapper
  - The owner class, `addLeft` / `addRight` with exceptions
  - Traversals: preorder, inorder, postorder
  - The standard library already contains trees: `std::set`, `std::map`

BINARY

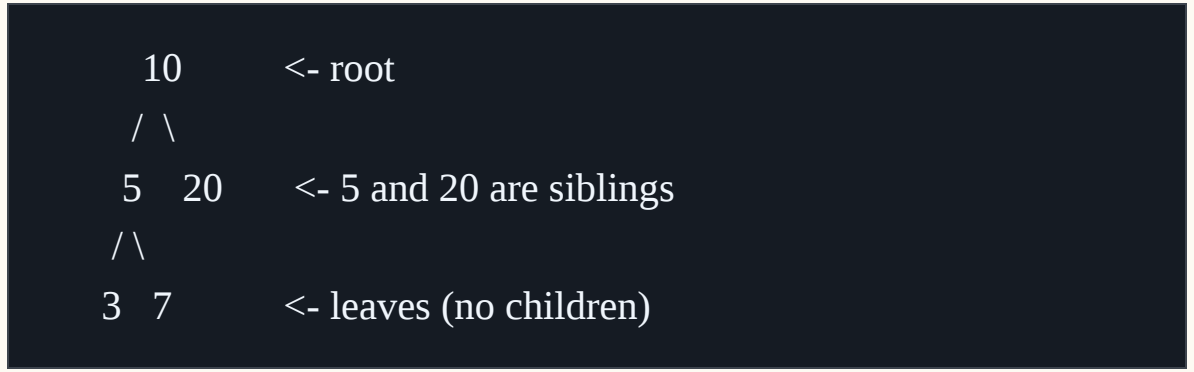
Trees

WHY TREES?

- Arrays and linked lists are **linear**: every element has one successor.
- A lot of real data is a **hierarchy**: folders and files, a company, an `HTML` page, an arithmetic expression, a family tree.
- A **tree** lets one element have **many** successors. The structure branches.

```
/                   Operations we want:
├── home                    - visit every element once
│   ├── anna                - find an element
│   └── ben                 - add and remove an element
└── usr                   - ask "how deep is it?"
    └── bin
```

Terminology



> Here: `5` is the parent of `3` and `7`. The depth of `7` is 2. The height of the tree is 2.

- **Node**: one element. **Edge**: the link between a parent and a child.
- **Root**: the only node without a parent.
- **Leaf**: a node without children.
- **Subtree**: a node together with everything below it.
- **Depth** of a node: edges from the root to it (`root` = 0).
- **Height** of a tree: the largest depth of any node.

BINARY TREE: TWO CHILDREN AT MOST

- In a **binary** tree every node has **at most two** children, and we **distinguish** them: the **left** child and the **right** child.
- The definition is **recursive**:

> A binary tree is either **empty**, or it is a **node** with a **left** binary tree and a **right** binary tree.

- The **empty tree** is the **base case**. In code it is a `nullptr`.
- Because of this definition, almost every algorithm on a tree is a short **recursive** function.

From DoubleNode to TreeNode

```
// last lecture: a linear structure
class DoubleNode
{
   private: int        value;
   private: DoubleNode* prevPtr;
   private: DoubleNode* nextPtr;
   private: friend class DoublyLinkedList;

   public: explicit DoubleNode(int value)
      : value(value), prevPtr(nullptr),
        nextPtr(nullptr) {}
   public: int getValue() const { return value; }
};
```

```
// today: a branching structure
class TreeNode
{
   private: int       value;
   private: TreeNode*  parentPtr;
   private: TreeNode*  leftPtr;
   private: TreeNode*  rightPtr;
   private: friend class BinaryTree;

   public: explicit TreeNode(int value)
      : value(value), parentPtr(nullptr),
        leftPtr(nullptr), rightPtr(nullptr) {}
   public: int getValue() const { return value; }
};
```

> Same pattern: private links, the owner class is a `friend`, the constructor sets all links to `nullptr`. Two links became **two children**, and we add a link **up**.

Why a Parent Pointer?

- Left and right pointers let us walk **down**. The `parentPtr` lets us walk **up**, towards the root, **without recursion**.
- We will need it for: the depth of a node, the path to the root, and later for removing a node.
- The price: **every** operation that links nodes must update **both** directions. One invariant to keep:

> If `p->leftPtr == c` (or `p->rightPtr == c`), then `c->parentPtr == p`.

- A leaf has two `nullptr` children. The root has a `nullptr` parent.

```
p           p->leftPtr   ---> c
^           c->parentPtr ---> p
|
c
```

EXCEPTIONS

THE C CONVENTION: RETURN CODES

```c
int divide(int a, int b, int *result)
{
    if (b == 0) return -1;      // error: nothing was computed
    *result = a / b;
    return 0;                   // 0 = success
}
int r;
if (divide(10, 0, &r) != 0) { /* handle the error */ }
```



- The **result** goes out through a pointer, the **error** through the return value. `0` means OK, `-1` means failure (like the exit code of a program).
- The same idea: `NULL` from `malloc` / `fopen`, `EOF`, and the global `errno`.

The Problem: Errors Travel by Hand

```
int parse(...)  { ...; if (divide(a, b, &r) != 0) return -1; ...; return 0; }
int load(...)   { ...; if (parse(...) != 0)       return -1; ...; return 0; }
int main(void)  { if (load(...) != 0) { /* finally handled here */ } }
```



- Every function in the chain must **check** and **pass the error up** manually.
- The caller can **ignore** the return code. The program compiles, runs, and continues with a **wrong value**, silently.
- The error code carries almost no information: `-1` does not say *what* went wrong.
- Constructors have **no return value** at all, so they cannot report a failure this way.

> C++ adds a second path out of a function, which **cannot be ignored**: the exception.

throw, try, catch

```cpp
#include <stdexcept>

int divide(int a, int b)
{
    if (b == 0) throw std::invalid_argument("divide: b is zero");
    return a / b;                          // the result is the return value again
}
try
{
    std::cout << divide(10, 0) << "\n";          // throws: the next line is skipped
    std::cout << "never printed\n";
}
catch (const std::invalid_argument& e)
{
    std::cout << "Error: " << e.what() << "\n";  // Error: divide: b is zero
}
```

- `throw` creates an exception **object** and leaves the function **immediately**.
- `try` marks the code we watch. `catch` handles one type of exception. `e.what()` returns the

WHAT HAPPENS WHEN NOBODY CATCHES?

```
main()        try { load(); }  catch (const std::exception& e) { ... }   <- lands HERE
  load()      parse();
    parse()   divide(10, 0);
      divide()  throw std::invalid_argument(...);                  <- starts HERE
```

- **Stack unwinding**: the program leaves `divide`, `parse`, `load` one by one, **destroying their local objects** (destructors run), until it finds a matching `catch`.
- No function in the middle needs any `if`. The error **jumps over them**.
- If **no** `catch` matches, the program calls `std::terminate` and **stops**. An error can no longer pass silently.

Standard Exception Types

Type	Header	Typical use
`std::exception`	`<exception>`	the **base class**; `what()` returns the message
`std::logic_error`	`<stdexcept>`	an error that the caller could have avoided
`std::invalid_argument`	`<stdexcept>`	an argument has a wrong value (derived from `logic_error`)
`std::out_of_range`	`<stdexcept>`	an index outside the valid range (`vector::at`)
`std::runtime_error`	`<stdexcept>`	an error visible only at run time: file, input, ...
`std::bad_alloc`	`<new>`	`new` could not get memory

- **Throw by value, catch by `const` reference**: `catch (const std::exception& e)`. Remember slicing from the last lecture: a copy of the base type would cut off the derived part.
- Several `catch` blocks are tried **top to bottom**. Put the **most specific** type first and `std::exception` last.

C++ Is Not Java or C#

	C++	Java / C#
Declared in the signature?	**No.** `noexcept` only says "never throws"	Java: checked exceptions (`throws`). C#: no
`finally`	**None.** Destructors clean up (RAII)	`finally`, `using`
What can be thrown	**Any type**. By convention: classes derived from `std::exception`	only `Throwable` / `Exception` classes
Null pointer, bad index	**Undefined behavior.** No exception, no check	always an exception
Stack trace	none built in	built in
Culture	exceptions for **rare** failures. Cheap when nothing is thrown, expensive when thrown	used for many ordinary errors

> The C++ philosophy: **you do not pay for what you do not use**. The language does not add checks that you did not ask for.

Example: [] Does Not Check, at() Does

```cpp
std::vector<int> v = {1, 2, 3};

v[10];        // undefined behavior: no check. Garbage, or a crash, or "it works"
v.at(10);     // throws std::out_of_range

try
{
    std::cout << v.at(10);
}
catch (const std::out_of_range& e)
{
    std::cout << "Error: " << e.what();     // the text depends on the compiler
}
```

- The standard library gives you **both**: the fast version for code that is **known** to be correct, and the checked one when you are not sure.
- The same is true in C: `strcpy`, `memcpy`, `qsort` trust that you gave them correct arguments.

Who Checks What?

Situation	Who is responsible	What the function does
The **caller broke the contract**: null pointer, index out of range, node from another tree	the **programmer**	documents the precondition. May check **cheap minimum**. Otherwise undefined behavior
Something **outside the caller's control**: no memory, file missing, bad user input	**nobody**	**throws** an exception

- A function is **not** a bodyguard. It does its job **if** the caller keeps the contract, and it checks only what is **cheap** and what **prevents an obvious crash**, e.g. `nullptr`.
- It **cannot** check everything. A `nullptr` is easy to detect, but a **dangling** or **garbage** pointer is not. No function can fully protect you from a segmentation fault.
- Exceptions are for situations that **should not happen in a normal run**, not for ordinary control flow like "end of the list".

NO FINALLY: DESTRUCTORS CLEAN UP

```cpp
void process()
{
    std::vector<int> data(1000);    // owned by an object: freed automatically
    int* raw = new int[1000];       // owned by nobody: must be freed by hand
    mayThrow();                     // if it throws here ...
    delete[] raw;                   // ... this line is skipped: LEAK
}                                   // data is destroyed during unwinding
```



- While the stack unwinds, the **destructors** of local objects run. `data` is released, `raw` is **not**.
- The C++ answer to `finally` is **RAII**: a resource (memory, file, lock) belongs to an **object**, and the destructor releases it.
- Rule: **do not call `new` in the middle of code that can throw**. Put the memory inside a class. The `BinaryTree` on the next slides is exactly that.

PUTTING IT

Together

The BinaryTree: the Owner of the Nodes



```
class BinaryTree
{
   private: TreeNode*    rootPtr;
   private: std::size_t  count;

   public: BinaryTree() : rootPtr(nullptr), count(0) {}
   public: ~BinaryTree() { destroy(rootPtr); }
   public: BinaryTree(const BinaryTree&) = delete;            // two owners would
   public: BinaryTree& operator=(const BinaryTree&) = delete;   // delete twice

   public: bool        empty() const   { return rootPtr == nullptr; }
   public: std::size_t size()  const   { return count; }
   public: TreeNode*   getRoot() const { return rootPtr; }

   public: TreeNode* setRoot(int value);
   public: TreeNode* addLeft (TreeNode* parent, int value);
   public: TreeNode* addRight(TreeNode* parent, int value);
   // traversals, depthOf, destroy: next slides
};
```

addLeft with Exceptions

```
TreeNode* BinaryTree::addLeft(TreeNode* parent, int value)
{
    if (parent == nullptr)                          // the cheap minimum
        throw std::invalid_argument("addLeft: parent is null");
    if (parent->leftPtr != nullptr)                 // a state we cannot repair
        throw std::logic_error("addLeft: left child already exists");

    TreeNode* node = new TreeNode(value);           // may throw std::bad_alloc
    node->parentPtr = parent;                       // link both directions
    parent->leftPtr = node;
    count++;
    return node;                                    // the caller can add children to it
}
```



- `addRight` is identical, with `rightPtr`. `setRoot` throws `std::logic_error` when a root already exists.
- If `new` throws, **nothing has been changed yet**: the tree stays valid.
- We do **not** check that `parent` belongs to **this** tree. That is the programmer's contract.

Using the Tree

```
BinaryTree t;
TreeNode* root = t.setRoot(10);
TreeNode* l    = t.addLeft(root, 5);
t.addRight(root, 20);
t.addLeft(l, 3);
t.addRight(l, 7);

try
{
    t.addLeft(root, 99);   // 10 has a left child
}
catch (const std::logic_error& e)
{
    std::cout << e.what() << "\n";
}
```

```
      10
     /  \
    5    20
   / \
  3   7
```

- Each `add...` returns the **new node**, so we can attach the next children to it.
- `std::invalid_argument` is derived from `std::logic_error`, so this `catch` handles **both** errors.

Traversals: Visiting Every Node

A **traversal** visits each node exactly once. Because a tree is recursive, the code is recursive too. The three classic orders differ only in **when we visit the node itself**:

Order	Steps	Result for our tree
preorder	node, left, right	`10 5 3 7 20`
inorder	left, node, right	`3 5 7 10 20`
postorder	left, right, node	`3 7 5 20 10`

> Look at the **inorder** result: it is **sorted**. This is not an accident: in our tree every left child is smaller and every right child is bigger than its parent. That is a **binary search tree**, the topic of the next lecture.

Inorder in C++

```cpp
public: std::vector<int> inorder() const
{
    std::vector<int> out;
    inorderFrom(rootPtr, out);
    return out;                              // the caller decides what to do with it
}
private: static void inorderFrom(const TreeNode* node, std::vector<int>& out)
{
    if (node == nullptr) return;                 // base case: the empty tree
    inorderFrom(node->leftPtr, out);                 // 1. the whole left subtree
    out.push_back(node->value);                      // 2. the node itself
    inorderFrom(node->rightPtr, out);                 // 3. the whole right subtree
}
```

```cpp
for (int x : t.inorder()) std::cout << x << " ";     // 3 5 7 10 20
```

> Preorder and postorder: **swap the three lines**. A public method with a private recursive helper is a typical pattern.

The Destructor Is a Postorder

```
private: static void destroy(TreeNode* node)
{
   if (node == nullptr) return;       // base case
   destroy(node->leftPtr);            // children first ...
   destroy(node->rightPtr);
   delete node;                       // ... the node last
}
```



- If we deleted the node **first**, we would lose the only pointers to its children: a **memory leak** of the whole subtree.
- The **same rule as in C**: free the inside first, then the outside.
- `~BinaryTree()` calls `destroy(rootPtr)`. The user of the tree never calls `delete` on a node.

The Parent Pointer at Work

```
public: int depthOf(const TreeNode* node) const
{
   if (node == nullptr) throw std::invalid_argument("depthOf: node is null");
   int depth = 0;
   while (node->parentPtr != nullptr)      // climb up to the root
   {
      node = node->parentPtr;
      depth++;
   }
   return depth;
}
```



```
t.depthOf(root);      // 0
t.depthOf(l);         // 1
```



⚒ No recursion and no extra memory: $T(n) = O(h)$, $S(n) = \Theta(1)$, where h is the height of the tree.

COST OF A TRAVERSAL

- Every node is visited **once**, so the time is $T(n) = \Theta(n)$.
- The recursion goes as deep as the tree is **high**, so the stack needs $S(n) = \Theta(h)$.
- The height h depends on the **shape** of the tree:

```
balanced:   h = Θ(log n)        degenerate ("a list"):   h = n - 1
       10                          1
      /  \                          \
     5    20                         2
    / \                              \
   3   7                              3 ...
```

- A degenerate tree is a **linked list** with extra pointers, and a deep recursion may overflow the stack. Keeping the tree **balanced** is the main problem of the next lectures.

THE STANDARD LIBRARY HAS TREES

```
#include <set>

std::set<int> s = {20, 5, 10, 3, 7};      // inserted in a random order
for (int x : s) std::cout << x << " ";    // 3 5 7 10 20: an inorder traversal
s.insert(15);
s.find(7) != s.end();                     // true, in O(log n)
```



- `std::set` and `std::map` are typically implemented as **balanced binary search trees** (red-black trees). Iterating over them is an **inorder** traversal, which is why the elements come out **sorted**.
- This is why `std::set` needs only `operator<` from its elements, the same operator as in the sorting part of the previous lecture.
- We write our own tree to **understand** these containers, not to replace them.

SUMMARY

- A **binary tree** is empty, or a node with a **left** and a **right** subtree. `TreeNode` has `leftPtr`, `rightPtr` and a `parentPtr`, and keeps one invariant: *child's parent is the node that points to it*.
- In **C** an error is a return value (`0` / `-1`) that can be ignored. In **C++** an error is an **exception** that cannot be ignored: `throw`, `try`, `catch`, stack unwinding. Throw by value, catch by `const&`.
- C++ is **not** Java or C#: no `throws`, no `finally` (RAII instead), and **no checks** you did not ask for.
- The **function** checks only the cheap minimum (e.g. `nullptr`). The **contract** is the programmer's. Exceptions are for what the caller cannot control.
- `BinaryTree` **owns** the nodes. Traversals are recursive, the destructor is a postorder, `std::set` is a tree.

NEXT LECTURE

- **Binary search tree**: `insert`, `find`, `remove`, and why they cost $\Theta(h)$.
- **Level-order traversal** with `std::queue`, and traversals without recursion.
- **Balance**: what happens with sorted input, and how `std::set` avoids it.
- Custom exception classes, and exceptions in constructors.

THANK

You